NEXUS-1 — Project Roadmap

Destination: a platform active in all operating modes

The goal of NEXUS-1 is a platform that can operate - actively and functionally - in each of its modes: **Training**, **Shadow**, **Advisory**, **Supervisory** and **Hybrid Digital Twin**, building out from the Phase 0 **Simulation** build. This roadmap treats those modes as the milestones, records the intended technology stack, and is honest about what is achievable through engineering alone and what additionally depends on partnership and certification.

> **GUIDING SAFETY PRINCIPLE (EVERY MODE, EVERY PHASE)**
>
> NEXUS-1 is a monitoring, analytics, optimisation and decision-support layer. In **no** mode does it control or bypass certified reactor safety systems, which remain hardware-isolated, deterministic and human-supervised. All AI and optimisation output is advisory; any connection to a real plant is read-only until - and unless - a formal assurance and certification path is completed.

The destination: operating modes as milestones

Each mode is a capability the platform should demonstrate. Two of them (Training, and the simulated baseline) need nothing but the software itself. Shadow, Advisory and Hybrid Digital Twin are equally buildable, but reach **production** only with read-only access to a real plant's data - an access and partnership question, not a coding one. Supervisory is the most demanding: the platform can be made supervisory-capable and proven in a simulated or hardware-in-the-loop environment, but real supervisory operation additionally requires formal assurance, certification and a utility/vendor partner. It is an organisational milestone as much as an engineering one.

Mode	What it means	What it takes
Simulation	Fully virtual plant; the Phase 0 build.	Done.
Training	Operator education and incident drills in a virtual plant.	Engineering only - fully achievable solo.
Shadow	Mirrors live plant telemetry with no control authority.	Engineering + a read-only data feed from a real plant.
Hybrid Twin	A simulation kept continuously in step with live data.	Engineering + the same live feed; builds on Shadow.
Advisory	AI/analytics recommendations to human operators.	Engineering; advisory only, validated on live/shadow data.
Supervisory	Integrates with and orchestrates control infrastructure, human-supervised.	Assurance + certification + a utility/vendor partner; proven in simulation/HIL first.

Technology stack

The production system targets a **.NET / C# microservices** architecture on Azure with an **Angular** front end:

- **Backend:** .NET (C#) microservices on ASP.NET Core; REST for UI/external APIs, gRPC service-to-service. (No Orleans.)
- **Front end:** Angular single-page application.

- **Real-time:** SignalR to push live telemetry, alarms and state to the client.
- **Relational data:** Microsoft SQL Server for configuration, master data and audit/compliance.
- **Messaging:** Azure Service Bus for events and commands (topics/subscriptions, sessions, dead-lettering); Azure Event Hubs (or Kafka) for the high-rate telemetry stream.
- **Time-series:** SQL Server (partitioning / columnstore) initially; a dedicated time-series store later if volume warrants.
- **Ops:** Docker and Kubernetes for high availability; OpenTelemetry for observability.

Phased delivery plan

Phase 0 - Simulation mode

DONE

Goal: Prove the concept, the experience, the domain understanding and authorship.

- Self-contained web console covering the full plant, driven by a live in-browser simulation.
- Genuine reactor point-kinetics physics and a real, interpretable reinforcement-learning optimiser.
- A connected-system design (a SCRAM ripples into incidents, compliance, inspection and ageing).
- Complete documentation set: this roadmap, the functional guide, glossary, brief and readme.

Phase 1 - Foundation

NEXT

Goal: Stand up the service skeleton and move the simulation server-side. (Enabler for every mode.)

- Create the .NET solution and a microservices skeleton, with CI/CD and containerisation.
- Provision SQL Server, Azure Service Bus and Event Hubs; define schema and message contracts.
- Build a Simulation/Twin service running the physics server-side, streamed via SignalR.
- Scaffold the Angular front end and connect it to the API and the live stream.

Phase 2 - Training mode (active)

PLANNED

Goal: Make the first real mode functional - end to end, with no plant required.

- Instructor station: define scenarios, inject faults and events, pause/resume and replay.
- Multi-user sessions (operators + instructor); recordable and reviewable runs.
- Persisted scenario library and session history in SQL Server.
- Outcome: Training is a working, demonstrable mode.

Phase 3 - Shadow + Hybrid Digital Twin (active)

PLANNED

Goal: Mirror live data read-only, and keep the twin synchronised to it.

- Read-only industrial-protocol gateway (OPC-UA first) ingesting live - or emulated - telemetry.
- Shadow mode: the console mirrors the source plant with strictly no control path.
- Hybrid Digital Twin: the model runs continuously alongside the feed, with divergence detection as an early-warning signal.
- Production use needs a real read-only feed (a partnership); until then, proven against an emulated plant.

Phase 4 - Advisory mode (active)

PLANNED

Goal: Turn analytics into trustworthy, explainable recommendations.

- Predictive-diagnostics and optimisation services consuming real/shadow history.
- Operator-facing recommendations with plain-language explanations and confidence - advisory only.
- Human-in-the-loop acceptance workflow and full audit of advice given and acted on.

Phase 5 - Supervisory mode (capability; deployment gated)

PLANNED

Goal: Be supervisory-capable and proven in simulation; gate real deployment behind assurance.

- Integration and orchestration with control infrastructure, validated in a simulated / hardware-in-the-loop rig.
- Operational-technology security throughout: zero-trust, segmentation, key management, MFA, intrusion detection.
- Formal verification of supervisory logic; redundancy, high availability and disaster recovery.
- Real supervisory operation only after certification and with a utility/vendor partner - and never controlling or bypassing certified safety systems.

> **A NOTE ON AMBITION AND HONESTY**
>
> Training, Shadow, Advisory and Hybrid Digital Twin are within reach of this project as built-and-proven software; what they need for the real world is access to plant data, not more code. Supervisory is the line where engineering meets regulation - the platform can be made ready and demonstrated, but live supervisory deployment is a certification-and-partnership milestone. Stating this plainly is what keeps the destination credible.

Appendix: indicative effort estimate

> **HOW TO READ THESE NUMBERS**
>
> These are rough order-of-magnitude figures - assume **+/- 50%**, widening for later phases - for **engineering build effort by one experienced .NET engineer**. They deliberately exclude what is not a coding problem: obtaining a real plant's read-only data feed, and the certification, assurance and partnership work in Phase 5. Those are organisational person-years, not solo work-hours, and are not estimated here. This appendix is indicative planning, not a commitment.

Phase	Scope	Work-hours	Confidence
0 - Simulation	Current build + documentation	~200-300 (invested)	done
1 - Foundation	Services skeleton, CI/CD, SQL Server + Service Bus + Event Hubs, physics to C#, Angular + SignalR	**200-350**	moderate
2 - Training	Instructor station, fault injection, replay, multi-user, persistence	**180-320**	moderate
3 - Shadow + Hybrid Twin	Read-only OPC-UA gateway, ingestion, shadow mirror, twin sync + divergence	**200-360**	lower
4 - Advisory	Diagnostics/optimisation services, explainable recommendations, accept + audit	**140-260**	lower
5 - Supervisory (eng. only)	Integration proven in simulation/HIL, OT security, formal verification, HA/DR	**250-500**	low
Contingency (~25%)		**~240-450**	-
Total (Phases 1-5 build)	Engineering capability, demonstrable in simulation	**~1,200-2,240**	rough

Phase 5's certification, regulatory assurance and utility-partner work is **not** included above and is not solo-estimable - realistically person-years inside an established organisation. The figure is only the engineering capability that could be built and demonstrated in simulation.

In calendar terms (build portion only):

- **Part-time, ~10 h/week:** roughly 2.5-4.5 years.
- **Serious part-time, ~20 h/week:** roughly 1.25-2.25 years.
- **Full-time, ~35 h/week:** roughly 8-14 months.

Two honest observations. The estimates are most trustworthy for Phases 1-2 (familiar work) and least trustworthy for 3-5, where unknowns - data assimilation, model quality, OT-security depth - can each double a phase. And the highest-value near-term target is reaching **Phase 2 (Training active)**: roughly 400-700 hours total from the current build, needing no external partner, which turns a demonstration into a working server-based product with real users. Planning is best anchored on that milestone rather than the full total.